

AI Practical Assignment

Natya Vidhan Biswas

B.Sc. (H) C.S. 25771

1. Write a PROLOG program to implement the family tree and demonstrate the family relationship.

```
parent(pam, bob).  
parent(tom, bob).  
parent(tom, liz).  
parent(bob, pat).  
parent(bob, ann).  
parent(pat, jim).  
parent(liz, rahul).
```

```
male(bob).  
male(tom).  
male(jim).  
male(rahul).
```

```
female(pat).  
female(pam).  
female(liz).  
female(ann).
```

```
grandparent(X, Z) :-  
    parent(X, Y),  
    parent(Y, Z).
```

```
father(X, Y) :-  
    male(X),  
    parent(X, Y).
```

```
mother(X, Y) :-  
    female(X),  
    parent(X, Y).
```

```
sibling(X, Y) :-  
    parent(Z, X),  
    parent(Z, Y),  
    X \= Y.
```

```
brother(X, Y) :-  
    male(X),  
    sibling(X, Y).
```

```
sister(X, Y) :-  
    female(X),  
    sibling(X, Y).
```

sibling(X, Y).

uncle(X, Y) :-
brother(X, Z),
parent(Z, Y).

aunt(X, Y) :-
sister(X, Z),
parent(Z, Y).

cousin(X, Y) :-
parent(P1, X),
parent(P2, Y),
sibling(P1, P2).

child(X, Y) :-
parent(Y, X).

son(X, Y) :-
male(X), child(X, Y).

daughter(X, Y) :-
female(X), child(X, Y).

grandfather(X, Y) :-
male(X), grandparent(X, Y).

grandmother(X, Y) :-
female(X), grandparent(X, Y).

grandson(X, Y) :-
male(X), grandparent(Y, X).

granddaughter(X, Y) :-
female(X), grandparent(Y, X).

nephew(X, Y) :-
male(X), parent(P, X), sibling(P, Y).

niece(X, Y) :-
female(X), parent(P, X), sibling(P, Y).

ancestor(X, Y) :-
parent(X, Y).

ancestor(X, Y) :-
parent(X, Z), ancestor(Z, Y).

```
descendant(X, Y) :-  
    ancestor(Y, X).
```

```
?- father(tom, bob).  
true.
```

```
?- mother(pam, bob).  
true .
```

```
?- sibling(bob, liz).  
true.
```

```
?- uncle(bob, rahul).  
true.
```

```
?- aunt(ann, jim).  
true .
```

```
?- cousin(jim, rahul).  
false.
```

```
?- cousin(ann, rahul).  
true.
```

```
?- grandfather(tom, pat).  
true .
```

```
?- grandson(jim, bob).  
true .
```

```
?- ancestor(tom, jim).  
true .
```

2. Write a PROLOG program to implement `conc(L1, L2, L3)` where L2 is the list to be appended with L1 to get the resulted list L3.

```
conc([], L, L) :- !.  
conc([H|T], L, [H|R]):-  
    conc(T, L, R).
```

```
?- conc([a,b], [c,d], R).  
R = [a, b, c, d].
```

3. Write a PROLOG program to implement `reverse(L, R)` where List L is original and List R is reversed list.

```
reverse(L, R):-  
    reverse(L, [], R).  
reverse([], Acc, Acc) :- !.  
reverse([H|T], Acc, R):-  
    reverse(T, [H|Acc], R).
```

```
?- reverse([1,2,3], R).  
R = [3, 2, 1].
```

4. Write a PROLOG program to calculate the sum of two numbers.

```
sum(X, Y, S):-  
    S is X + Y.
```

```
?- sum(10, 20, S).  
S = 30.
```

5. Write a PROLOG program to implement `max(X, Y, M)` so that M is the maximum of two numbers X and Y.

```
max(X, Y, M):-  
    X > Y -> M = X; M = Y.
```

```
?- max(5, 8, M).  
M = 8.
```

6. Write a program in PROLOG to implement factorial (N, F) where F represents the factorial of a number N.

```
factorial(1, 1) :- !.  
factorial(N, F):-  
    N1 is N - 1,  
    factorial(N1, F1),  
    F is N * F1.
```

```
?- factorial(6, F).  
F = 720.
```

7. Write a program in PROLOG to implement generate_fib(N,T) where T represents the Nth term of the Fibonacci series.

```
generate_fib(0, 0) :- !.  
generate_fib(1, 1) :- !.  
  
generate_fib(N, T) :-  
    N > 1,  
    N1 is N - 1,  
    N2 is N - 2,  
    generate_fib(N1, T1),  
    generate_fib(N2, T2),  
    T is T1 + T2.
```

```
?- generate_fib(7, T).  
T = 13.
```

8. Write a PROLOG program to implement power (Num, Pow, Ans) : where Num is raised to the power Pow to get Ans.

```
power(_, 0, 1) :- !.  
power(N1, 1, N1) :- !.  
power(N1, N2, R):-  
    SN2 is N2 - 1,  
    power(N1, SN2, R1),  
    R is R1 * N1.
```

```
?- power(2, 10, R).
```

R = 1024.

9. PROLOG program to implement multi (N1, N2, R) : where N1 and N2 denotes the numbers to be multiplied and R represents the result.

```
multi(N1, 1, N1) :- !.  
multi(N1, N2, R):-  
    SN2 is N2 - 1,  
    multi(N1, SN2, R1),  
    R is R1 + N1.
```

```
?- multi(7, 8, R).  
R = 56.
```

10. Write a PROLOG program to implement memb(X, L): to check whether X is a member of L or not.

```
memb(X, [X|_]) :- !.  
memb(X, [_|T]):-  
    memb(X, T).
```

```
?- memb(b, [a,b,c]).  
true.
```

11. Write a PROLOG program to implement sumlist(L, S) so that S is the sum of a given list L.

```
sumlist([], 0) :- !.  
sumlist([H|T], S):-  
    sumlist(T, S1),  
    S is H + S1.
```

```
?- sumlist([5,5,5], S).  
S = 15.
```

12. Write a PROLOG program to implement two predicates `evenlength(List)` and `oddlength(List)` so that they are true if their argument is a list of even or odd length respectively.

```
evenlength([], N) :-  
    0 is N mod 2, !.  
evenlength(_|T, N) :-  
    N1 is N + 1,  
    evenlength(T, N1).  
evenlength(R) :-  
    evenlength(R, 1).
```

```
oddlength([], N) :-  
    1 is N mod 2, !.  
oddlength(_|T, N) :-  
    N1 is N + 1,  
    oddlength(T, N1).  
oddlength(R) :-  
    oddlength(R, 1).
```

```
?- evenlength([1,2]).  
true.
```

```
?- oddlength([1,2,3,4,5]).  
true.
```

13. Write a PROLOG program to implement `maxlist(L, M)` so that M is the maximum number in the list.

```
maxlist([X], X) :- !.  
maxlist([H|T], M) :-  
    maxlist(T, NM),  
    (H > NM -> M = H; M = NM).
```

```
?- maxlist([1,5,3], M).  
M = 5.
```


14. Write a PROLOG program to implement insert(I, N, L, R) that inserts an item I into Nth position of list L to generate a list R.

```
insert(I, 1, L, [I|L]) :- !.  
insert(I, N, [H|T], [H|R]) :-  
    N > 1,  
    N1 is N - 1,  
    insert(I, N1, T, R).
```

```
?- insert(z, 1, [a,b], R).  
R = [z, a, b].
```

15. Write a PROLOG program to implement delete(N, L, R) that removes the element on Nth position from a list L to generate a list R.

```
del(1, [_|L], L) :- !.  
del(N, [H|T], [H|R]) :-  
    N > 1,  
    N1 is N - 1,  
    del(N1, T, R).
```

```
?- del(3, [a,b,c,d], R).  
R = [a, b, d].
```

Extra Programs

1. Length of a List

`len([], 1) :- !.`

`len([_|T], L):-`

`len(T, L1),`

`L is L1 + 1.`

`?- len([a,b,c], L).`

`L = 3.`

2. Greatest Common Denominator

`gcd(X, 0, X) :- X > 0, !.`

`gcd(X, Y, G) :-`

`Y > 0,`

`R is X mod Y,`

`gcd(Y, R, G).`

`?- gcd(54, 24, G).`

`G = 6.`